

Git HTTPS Access When SSH Is Blocked by Netskope

Purpose

Netskope blocks SSH traffic in some environments. If Git clone, pull, or push operations fail when using SSH remotes, use HTTPS repository URLs instead.

This commonly affects remotes that look like:

```
git@github.com:ORG/repo.git  
git@bitbucket.org:ORG/repo.git  
ssh://git@github.com/ORG/repo.git  
ssh://git@bitbucket.org/ORG/repo.git
```

Use HTTPS remotes instead:

```
https://github.com/ORG/repo.git  
https://bitbucket.org/ORG/repo.git
```

Clone a Repository Over HTTPS

Use the HTTPS repository URL when cloning:

```
git clone https://github.com/ORG/repo.git
```

For Bitbucket:

```
git clone https://bitbucket.org/bayview-asset-management/scenarios_mfe.git
```

Convert an Existing Repository From SSH to HTTPS

From inside the local repository, update the **origin** remote:

```
git remote set-url origin URI
```

Replace **URI** with the HTTPS repository URL.

For GitHub:

```
git remote set-url origin https://github.com/ORG/repo.git
```

For Bitbucket:

```
git remote set-url origin https://bitbucket.org/bayview-asset-  
management/scenarios_mfe.git
```

Verify the Remote

Confirm that `origin` now uses HTTPS:

```
git remote -v
```

Expected format:

```
origin https://github.com/ORG/repo.git (fetch)  
origin https://github.com/ORG/repo.git (push)
```

or:

```
origin https://bitbucket.org/bayview-asset-management/scenarios_mfe.git (fetch)  
origin https://bitbucket.org/bayview-asset-management/scenarios_mfe.git (push)
```

Pull and Push

After the remote is set to HTTPS, normal Git commands should use HTTPS automatically:

```
git pull  
git push
```

If authentication is required, sign in with the approved Git credential flow for the platform.

Credential Helper Troubleshooting

Users may run into credential helper issues after switching from SSH to HTTPS. Common symptoms include repeated login prompts, authentication failures, Git using the wrong account, or Git continuing to fail even after the remote URL was changed correctly.

Common causes include:

- Git is using old or incorrect cached credentials from Windows Credential Manager or Git Credential Manager.
- The remote URL includes an old username, such as `https://olduser@bitbucket.org/ORG/repo.git`.
- GitHub or Bitbucket rejects password authentication and requires SSO, browser-based sign-in, an app password, or a personal access token depending on company policy.
- Multiple credential helpers are configured, such as `manager`, `manager-core`, `wincred`, or `store`, and Git is using an unexpected one.
- Netskope or TLS inspection issues are being mistaken for credential failures.

Check the configured remote URL:

```
git remote -v
```

Check which credential helpers Git is using:

```
git config --show-origin --get-all credential.helper
```

If Git is using stale GitHub credentials, clear them from Git Credential Manager:

```
@  
protocol=https  
host=github.com  
  
"@ | git credential-manager erase
```

If Git is using stale Bitbucket credentials, clear them from Git Credential Manager:

```
@  
protocol=https  
host=bitbucket.org  
  
"@ | git credential-manager erase
```

On Windows, users can also remove stale entries from **Credential Manager** under **Windows Credentials**. Look for entries such as:

```
git:https://github.com  
git:https://bitbucket.org
```

After clearing stale credentials, retry the Git operation:

```
git pull  
git push
```